



School:Campus:

Academic Year: Subject Name: Subject Code:

Semester: Program: Branch: Specialization:

Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment: Stake Your Claim – Proof of Stake Simulation

***Coding Phase: Pseudo Code / Flow Chart / Algorithm**

Theory :

Proof of Stake (PoS) is a consensus algorithm used in block chain networks where the right to validate a block is determined by the validator's stake — the amount of cryptocurrency they lock into the network.

Unlike **Proof of Work (PoW)**, PoS does not require heavy computation. Validators with larger stakes have higher chances of being selected, making the system energy-efficient and eco-friendly.

- **Stake:** Amount of cryptocurrency locked by a validator.
- **Validator:** Node responsible for creating or validating new blocks.
- **Reward:** Incentive for validators to act honestly.
- **Slashing:** Penalty for malicious or invalid behavior.

Algorithm :

1. Deploy Proof of Stake smart contract on Ethereum test network.
2. Users stake Ether to become validators.
3. Each validator's stake is recorded on the blockchain.
4. A validator is selected randomly, weighted by stake size.
5. The selected validator is rewarded.
6. Validators can withdraw their stake anytime.
7. End

*Software used :

- Vs code
- Remix IDE (Online)
- Solidity v0.8.19
- MetaMask (Injected Provider)
- Ethereum Test Network (Sepolia)
- Etherscan (Testnet)

Page No.....

*** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.**

* Testing Phase: Compilation of Code (error detection)

NO ERROR

* Implementation Phase: Final Output (no error)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract ProofOfStake {
    mapping(address => uint256) public stakes;
    address[] public validators;
    uint256 public totalStake;
    uint256 public reward = 0.001 ether; // smaller reward for testing
    uint256 public minStake = 0.001 ether; // set minimum stake limit

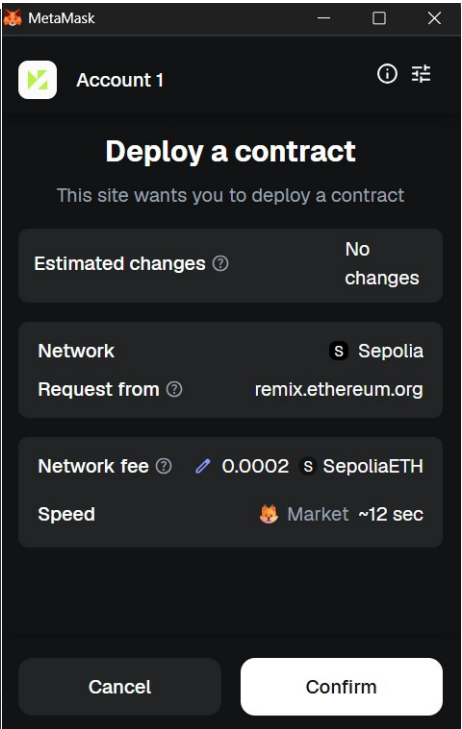
    event Staked(address indexed validator, uint256 amount);
    event ValidatorSelected(address indexed validator, uint256 reward);

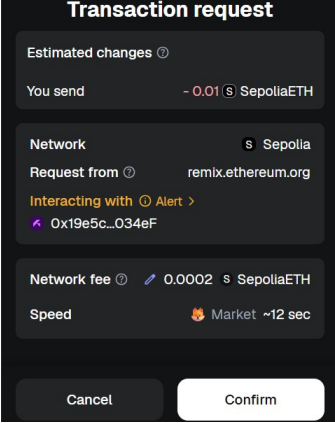
    function stake() external payable {
        require(msg.value >= minStake, "Stake must be at least 0.001 ETH");
        if (stakes[msg.sender] == 0) validators.push(msg.sender);
        stakes[msg.sender] += msg.value;
        totalStake += msg.value;
        emit Staked(msg.sender, msg.value);
    }

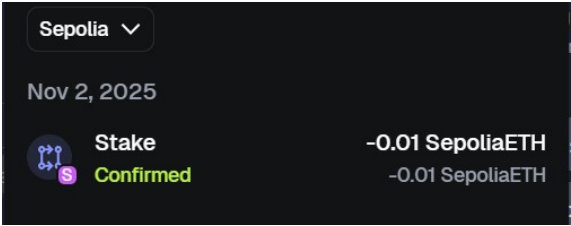
    function selectValidator() public view returns (address) {
        require(totalStake > 0, "No stakes yet");
        uint256 rand = uint256(keccak256(abi.encodePacked(block.timestamp, block.prevrandao))) % totalStake;
        uint256 sum = 0;
        for (uint i = 0; i < validators.length; i++) {
            sum += stakes[validators[i]];
            if (rand < sum) return validators[i];
        }
        return validators[validators.length - 1];
    }

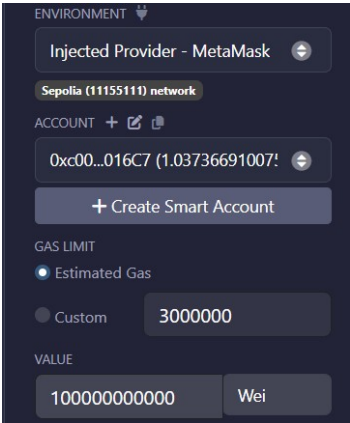
    function proposeBlock() external {
        address validator = selectValidator();
        payable(validator).transfer(reward);
        emit ValidatorSelected(validator, reward);
    }

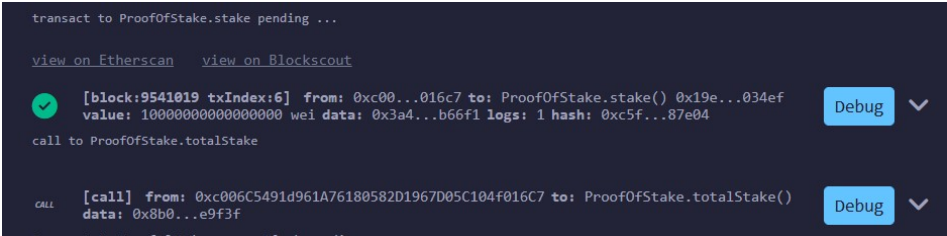
    receive() external payable {}
}
```











* Observation :

- Validators with higher stakes were selected more often, demonstrating stake-based probability.
- Gas fees were minimal since MetaMask used test ETH.
- Real-time updates could be tracked on Etherscan Testnet.
- PoS ensured fair and energy-efficient consensus without mining.

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student :

Name :

Signature of the Faculty :

Regn. No. :

Page No.....

**** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used***